

Tầng 0 và Tầng 1 trong VCuboidFIT

Bản chất, kỹ thuật, kiến trúc: tài liệu chuẩn bị thuyết trình

08/10/2026

1. Nói trong 30 giây

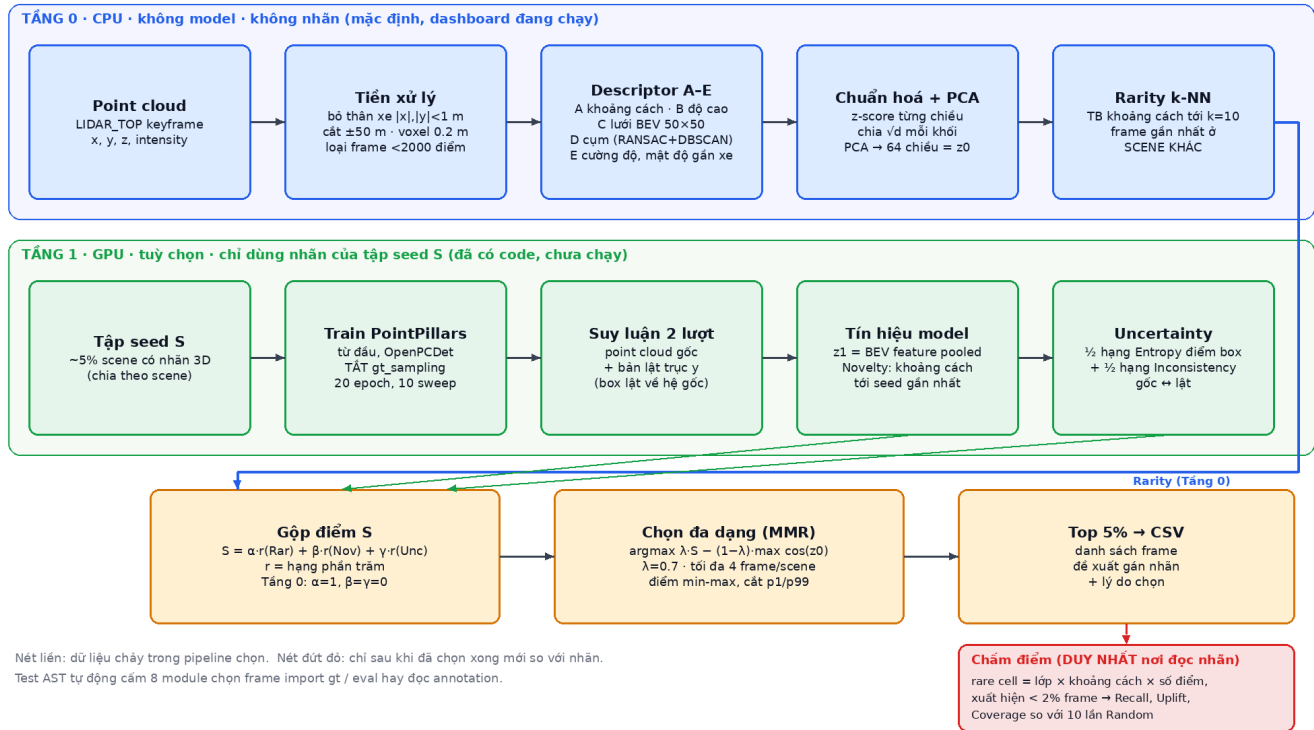
“Gán nhãn 3D rất đắt, nên chỉ gán được khoảng 5% dữ liệu. VCuboidFIT chấm điểm từng frame LiDAR theo hai tầng. **Tầng 0** chỉ nhìn *hình dạng* đám mây điểm: frame nào nằm ở vùng thưa, ít frame giống nó, thì được coi là hiếm. **Tầng 1** hỏi thêm *một model 3D*: frame nào model thấy lạ hoặc phân vân. Sau đó hệ thống gộp điểm và chọn 5% vừa có điểm cao vừa đa dạng. Nhấn thật chỉ dùng ở bước cuối, để chấm xem mình chọn có tốt hơn ngẫu nhiên không.”

Bốn khái niệm cần tách bạch (hội đồng rất hay hỏi):

Khái niệm	Câu hỏi nó trả lời	Ở đâu trong hệ thống
Rare (hiếm)	Frame này có ít frame giống nó trong dữ liệu không?	Tầng 0: Rarity k-NN
Novel (lạ)	Frame này có khác những gì model đã học không?	Tầng 1: Novelty
Uncertain (không chắc)	Model có phân vân khi dự đoán frame này không?	Tầng 1: Entropy + Inconsistency
Hard (khó)	Model có dự đoán sai frame này không?	Không dùng để chọn , vì muốn biết sai thì phải có nhãn

2. Kiến trúc tổng thể

Kiến trúc chọn 5% frame LiDAR: VCuboidFIT



Kiến trúc tổng thể: Tầng 0, Tầng 1, gộp điểm, MMR và vùng chấm điểm

Ba vùng truy cập nhân, đây là xương sống chống rò rỉ (leakage):

- Vùng không nhân**: toàn bộ Tầng 0, bước gộp điểm và MMR. Một test tự động đọc mã nguồn (AST) cấm 8 module này import `gt / eval` hay chạm vào annotation (`tests/lidar/test_no_leakage_lidar.py`).
- Vùng nhân seed**: chỉ Tầng 1, chỉ nhân của tập **S** (khoảng 5% số scene), để huấn luyện PointPillars.
- Vùng chấm điểm**: chỉ `gt.py` (đọc annotation) và `eval.py` (tính chỉ số). Chạy **sau khi** đã chọn xong.

Chia dữ liệu theo scene, không theo frame (`c4/lidar/splits.py`): S (seed) / V (tune tham số) / P (pool để chọn) / T (test, là tập val chính thức của nuScenes). Các frame liên nhau trong một scene rất giống nhau, nên nếu chia theo frame thì tập train và tập test sẽ “nhìn thấy nhau”.

Phân lớp phần mềm: Web Next.js (giao diện) → API FastAPI (`service/`) → engine Python `c4.lidar`. Tầng 1 chạy trong Docker riêng có GPU (OpenPCDet), engine chỉ đọc file kết quả `t1/signals.parquet`.

3. Tầng 0: hiếm về hình học, không model, không nhân

3.1 Bản chất

Ý tưởng: **mỗi frame được biến thành một điểm trong không gian đặc trưng. Frame nằm ở vùng thưa (xa các frame khác) là frame hiếm.** Đây là cách ước lượng mật độ phi

tham số: khoảng cách tới k láng giềng gần nhất càng lớn thì mật độ ở đó càng thấp. Ý tưởng này đã có từ lâu trong phát hiện ngoại lai (k-NN outlier, Ramaswamy et al., SIGMOD 2000).

Vì sao cần Tầng 0: nó chạy trên **CPU, không cần model và không cần nhãn**, nên dùng được ngay cả khi chưa có gì, kể cả với dữ liệu hoàn toàn mới.

3.2 Các bước kỹ thuật

Bước 1: Tiền xử lý (pcd.py)

- Đọc `.pcd.bin` (float32 × 5: x, y, z, intensity, ring).
- Bỏ điểm trên thân xe ($|x|, |y| < 1$ m), giống `remove_close` của nuscenes-devkit.
- Cắt vùng ±50 m.
- Voxel downsample 0.2 m (giữ một điểm mỗi ô, có tính tất định) để phân cụm nhanh.
- Frame còn dưới 2000 điểm bị loại (đánh dấu `few_points`).

Bước 2: Descriptor 5 khối A-E (descriptor.py)

Khối	Đo cái gì	Kỹ thuật	Ý nghĩa nghiệp vụ
A. Khoảng cách	Histogram khoảng cách ngang từ xe tới điểm (mốc 0, 5, 10, 15, 20, 30, 40, 50, 70 m) + log số điểm	Histogram chuẩn hoá	Cảnh trống trải hay chật hẹp, có bị che chắn gần không
B. Độ cao	Histogram trục z từ −3 m đến 5 m, bước 0.5 m	Histogram	Có vật cao (xe tải, cây, cầu) hay thấp (dốc, gờ)
C. Bố cục BEV	Lưới 50 × 50 ô 2 m, đếm điểm mỗi ô (log)	Ảnh mật độ nhìn từ trên	Hình dạng đường: ngã tư, đường cong, bãi đỗ
D. Cụm vật thể	Số cụm, phân bố kích thước cụm và diện tích BEV, số cụm nhỏ ở xa (≤ 20 điểm, > 30 m)	RANSAC tìm mặt đất (60 vòng, ngưỡng 0.25 m) → bỏ mặt đất → DBSCAN (eps 0.6 m, min 8 điểm)	Đông hay vắng vật thể, có nhiều vật nhỏ ở xa khó thấy không
E. Cảm biến	Tỉ lệ điểm không phải mặt đất trong bán kính 3 m, trung bình và độ lệch cường độ phản xạ	Thống kê	Có vật sát xe, bề mặt ướt hoặc phản xạ lạ (mưa, kính)

RANSAC (Fischler & Bolles, 1981): chọn ngẫu nhiên 3 điểm, dựng mặt phẳng, đếm số điểm khớp, rồi lặp lại và giữ mặt phẳng tốt nhất. Code chỉ nhận mặt phẳng gần nằm ngang (ngiêng dưới khoảng 37°). *DBSCAN* (Ester et al., KDD 1996): gom các điểm đứng gần nhau thành cụm mà không cần biết trước số cụm. Mỗi cụm xấp xỉ một vật thể.

Bước 3: Chuẩn hoá và PCA (descriptor.embed)

1. **z-score từng chiều** để các đơn vị khác nhau (mét, số đếm, cường độ) về cùng thang.
2. **Chia mỗi khối cho $\sqrt{\text{số chiều của khối}}$** . Khối C có 2500 chiều, khối E chỉ có 3. Nếu không chia, khối C sẽ lấn át toàn bộ khoảng cách. Sau bước này mỗi khối đóng góp gần như ngang nhau.

3. **PCA còn 64 chiều**, gọi là vector **z0**. PCA nén thông tin và bỏ chiều nhiễu/trùng lặp, giúp k-NN ổn định hơn. PCA được fit trên chính tập đang chọn; việc này hợp lệ vì PCA không dùng nhãn.

Bước 4: Rarity k-NN khác scene (`score.rarity`)

$$\text{Rar}(f) = \frac{1}{k} \sum_{j \in \text{kNN}(f), \text{scene}(j) \neq \text{scene}(f)} \| z_0(f) - z_0(j) \|, \quad k = 10$$

- **Chỉ tìm láng giềng ở scene khác.** Hai frame cách nhau 0.5 giây trong cùng một scene gần như giống hệt nhau. Nếu cho phép láng giềng cùng scene, frame nào cũng có láng giềng rất gần và trông “không hiếm”, và độ hiếm thật bị che mất.
- Frame hỏng vẫn được tính điểm để hiển thị, nhưng không được làm láng giềng cho frame khác.
- Tính theo khối 2048 dòng, nên chạy được với dữ liệu cỡ trainval.

Câu nói thuyết trình:

“Tầng 0 giống như xếp tất cả frame lên một tấm bản đồ theo hình dạng của chúng. Frame đứng một mình ở góc bản đồ, xa mọi frame của các cảnh khác, chính là frame hiếm.”

3.3 Điểm mạnh và giới hạn (nói trước khi bị hỏi)

Điểm mạnh	Giới hạn
Không cần GPU, model hay nhãn; chạy được ngay trên dữ liệu mới	“Hiếm về hình học” chưa chắc trùng với “hiếm về loại vật thể”. Một đĩa trẻ và một cột điện nhỏ có point cloud tương tự nhau
Có thể giải thích: biết khối nào làm frame lạ (có sẵn ablation bỏ từng khối)	Nhạy với nhiễu cảm biến: frame lỗi cũng có thể “lạ”
Tất định, tái lập được (seed cố định, dấu PCA cố định)	Descriptor do người thiết kế, không tự học

4. Tầng 1: hỏi ý kiến một model 3D

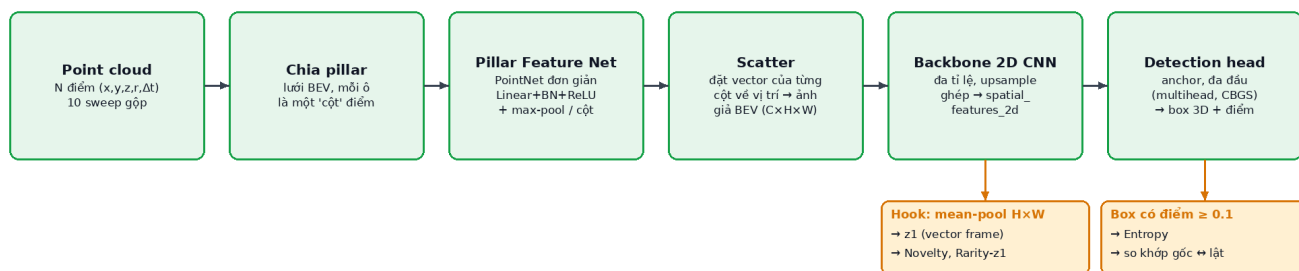
4.1 Bản chất

Tầng 0 hỏi “*dữ liệu có ít frame giống frame này không?*”. Tầng 1 hỏi thêm “**model** có thấy *frame này lạ hoặc khó đoán không?*”. Hai câu hỏi khác nhau: một frame có thể bình thường về hình học nhưng model vẫn phân vân, ví dụ vì vật thể bị che hoặc ở xa.

Để không rò rỉ nhãn, model được **huấn luyện từ đầu chỉ trên tập seed S**. Hệ thống **không dùng checkpoint công khai** của nuScenes, vì các checkpoint đó đã được huấn luyện trên nhãn của toàn bộ train, tức là đã “nhìn thấy” pool.

4.2 Model PointPillars

PointPillars (Lang et al., CVPR 2019) và chỗ VCuboidFIT lấy tín hiệu



PointPillars và vị trí lấy tín hiệu

PointPillars (Lang et al., CVPR 2019) là detector 3D cho LiDAR, nhanh và phổ biến:

1. **Chia pillar:** chia mặt phẳng BEV thành lưới, mỗi ô là một “cột” (pillar) chứa các điểm rơi vào đó.
2. **Pillar Feature Net:** một PointNet đơn giản (Linear + BatchNorm + ReLU, rồi max-pool) biến các điểm trong mỗi cột thành một vector.
3. **Scatter:** đặt vector của từng cột về đúng vị trí, tạo thành một **ảnh giả nhìn từ trên** ($C \times H \times W$).
4. **Backbone 2D CNN:** xử lý ảnh giả đó như ảnh thường, ở nhiều tỉ lệ.
5. **Detection head:** dự đoán box 3D (x, y, z, dài, rộng, cao, góc quay, vận tốc) kèm điểm tin cậy cho từng lớp.

Cấu hình trong code (`tier1/train_seed.py`): `OpenPCDet cbgs_pp_multihead.yaml`, gộp 10 sweep, 20 epoch.

- **Tắt `gt_sampling`.** Kỹ thuật tăng cường này dán vật thể lấy từ cơ sở dữ liệu nhãn của *toàn bộ* train vào cảnh, tức là đưa nhãn ngoài S vào model. Code sẽ dừng với mã lỗi nếu cấu hình còn `gt_sampling`.
- **Tự tắt CBGS** (cân bằng lớp, Zhu et al. 2019) khi tập seed thiếu lớp nào đó.
- **Khoá chạy** (`run_key`) bấm tập train, sweep, epoch và CBGS, để không bao giờ resume nhầm checkpoint của cấu hình khác.

4.3 Ba tín hiệu lấy từ model

(a) Vector `z1` và Novelty: “lạ với model” (`infer_t1.py`, `score.novelty`)

- Gắn một *hook* vào đầu ra của backbone 2D, lấy trung bình theo $H \times W$. Kết quả là một vector mô tả frame **theo cách model nhìn**, gọi là `z1`.
- $Nov(f) = \min_{s \in S} \|z_1(f) - z_1(s)\|$: khoảng cách tới frame seed gần nhất. Lớn nghĩa là model chưa từng thấy frame nào giống vậy.

(b) Entropy: “model phân vân về từng box” (`uncertainty.entropy`)

- Với mỗi box có điểm $p \geq 0.1$: $H(p) = -p \log p - (1-p) \log (1-p)$. Giá trị này lớn nhất khi $p = 0.5$, tức là model “nửa tin nửa ngờ”.
- $Ent(f) =$ trung bình H trên các box của frame. Frame không có box thì $Ent = 0$.

(c) Inconsistency: “model trả lời không nhất quán” (`uncertainty.inconsistency`)

- Chạy model **hai lần**: một lần trên point cloud gốc, một lần trên bản **lật trục y**. Box dự đoán trên bản lật được lật ngược về hệ gốc ($y \rightarrow -y$, góc quay $\rightarrow -\text{góc quay}$, $vy \rightarrow -vy$).

- Ghép box giữa hai lượt theo điểm giảm dần: cùng lớp, tầm BEV cách nhau dưới 1 m.
- $\text{Inc}(f) = 1 - \frac{\text{số cặp ghép được}}{\max(n_{\text{góc}}, n_{\text{lật}})}$. Một model tự tin sẽ cho kết quả giống nhau khi ta chỉ soi gương cảnh.
- Ý tưởng “dự đoán không nhất quán dưới phép biến đổi thì đáng gán nhãn” có trong active learning dựa trên tính nhất quán (ví dụ Gao et al., ECCV 2020).

Gộp thành Uncertainty: $\text{Unc}(f) = \frac{1}{2}r(\text{Ent}) + \frac{1}{2}r(\text{Inc})$, trong đó r là hạng phần trăm.

Câu nói thuyết trình:

“Tầng 1 cho một model học trên tập seed nhỏ rồi hỏi nó ba câu: frame này có giống cái gì em đã học không, em có chắc về từng hộp không, và nếu soi gương cảnh thì em có trả lời giống lần trước không.”

4.4 Trạng thái hiện tại (phải nói trung thực)

- **Dashboard đang chạy Tầng 0.** Tầng 1 đã có đủ code (huấn luyện, suy luận, tín hiệu, Docker, notebook Kaggle/Colab) nhưng **chưa chạy**, nên trên giao diện bị khoá với ghi chú “Chưa có model seed” (`pipeline.tier_available`).
- Khi chỉ có Tầng 0, **4 thể chiến lược cho kết quả giống nhau**, vì code chỉ dùng preset khi `tier = 1` (`params.resolve`).

5. Gộp điểm và chọn 5%

5.1 Gộp bằng hạng phần trăm

$$S(f) = \alpha r(\text{Rar}) + \beta r(\text{Nov}) + \gamma r(\text{Unc})$$

- Dùng **hạng phần trăm** (0–1) thay vì giá trị thô, vì ba tín hiệu có đơn vị và thang khác hẳn nhau. Hạng đưa chúng về cùng thang mà không cần giả định phân phối.
- Tầng 0: $\alpha = 1, \beta = \gamma = 0$. Các preset của Tầng 1: Cân bằng (0.5, 0.25, 0.25), Hiếm trước (0.8, 0.1, 0.1), Khó với model (0.2, 0.3, 0.5), Kích bản an toàn (0.6, 0, 0.4) (`configs/lidar.yaml`).

5.2 Chọn đa dạng bằng MMR

Nếu chỉ lấy top 5% điểm cao nhất, danh sách dễ dồn vào một khoảng khắc (ví dụ 10 frame liên tiếp ở cùng một ngã tư). **MMR** (Maximal Marginal Relevance, Carbonell & Goldstein, SIGIR 1998) chọn tham lam từng frame:

$$f^* = \arg \max_f [\lambda \cdot \text{điểm}(f) - (1 - \lambda) \cdot \max_{g \in \text{đã chọn}} \cos(Z_0(f), Z_0(g))]$$

- $\lambda = 0.7$ (mức “Vừa”); Thấp = 0.9, Cao = 0.5. $\lambda = 1$ nghĩa là tắt đa dạng.
- **Quota tối đa 4 frame mỗi scene.** Nếu không đủ chỗ cho ngân sách, quota tự nâng kèm cảnh báo.
- Điểm đưa vào MMR được **chuẩn hoá min-max** trên giá trị Rar thô (cắt phân vị 1/99) thay vì dùng hạng. Hạng làm mất biên độ: frame hiếm vượt trội và frame hiếm vừa trông gần

như nhau, khiến MMR dễ loại nhầm. Quyết định này được ghi lại trong config (Q4, Đ15), và hệ thống luôn báo cáo kèm phương án cũ để đối chứng.

5.3 Baseline để so sánh

Baseline	Bản chất	Để chứng minh điều gì
Random (10 seed)	Chọn ngẫu nhiên	Mức sàn
Random có quota	Ngẫu nhiên, tối đa m frame/scene	Lợi ích không chỉ đến từ việc trải đều scene
Top-k theo Rar	Không đa dạng	Giá trị của MMR
Coreset k-center (Sener & Savarese, ICLR 2018)	Chỉ đa dạng, không điểm hiếm	Giá trị của điểm hiếm
Oracle	Chọn bằng nhãn	Cận trên tham chiếu, không phải phương pháp

6. Chấm điểm: nơi duy nhất dùng nhãn

- **Rare cell** (gt.py): mỗi vật thể thuộc một ô = lớp × khoảng cách (0-20 / 20-40 / >40 m) × số điểm LiDAR (≤ 5 / 6-20 / >20). Ô xuất hiện ở dưới $\tau = 2\%$ số frame là ô hiếm. Frame chứa ít nhất một ô hiếm là **hiếm thật**. Cách nhóm lớp hiếm dựa trên LT3D (Peri et al., CoRL 2022).
- **Chỉ số**: Recall@5%, nRecall, Uplift (so với tỉ lệ kỳ vọng của Random), Coverage (độ phủ các ô hiếm), Scene-Recall cho mưa/đêm, Redundancy (frame chọn cách nhau ≤ 2 giây).
- **Quy trình thí nghiệm** (experiment.py): tune 48 cấu hình $k \times \lambda \times m$ **chỉ trên V**, chấm **một lần trên P**, CI 95% bằng bootstrap theo scene (lấy 80% scene, không hoàn lại), ablation bỏ từng khối A-E. Tiêu chí “hơn Random”: cận dưới CI > trung bình + 2 độ lệch chuẩn của Random.

7. Gợi ý bố cục slide

Slide	Nội dung	Hình
1	Bài toán: 5% ngân sách, Random bỏ sót frame hiếm	Ảnh dashboard
2	Rare $\neq$ Novel $\neq$ Uncertain $\neq$ Hard	Bảng mục 1
3	Kiến trúc 2 tầng + 3 vùng nhãn	Sơ đồ mục 2
4	Tầng 0: descriptor A-E $\rightarrow$ PCA $\rightarrow$ k-NN khác scene	Bảng khối A-E + công thức Rar
5	Tầng 1: PointPillars trên seed $\rightarrow$ Novelty, Entropy, Inconsistency	Sơ đồ PointPillars
6	Gộp hạng + MMR + quota	Công thức MMR
7	Chấm điểm và chống leakage	Rare cell + test AST
8	Trạng thái và bước tiếp theo	Tầng 0 chạy, Tầng 1 chờ GPU

8. Câu hỏi kỹ thuật hay gặp

Câu hỏi	Trả lời ngắn
Vì sao không dùng model pretrained cho nhanh?	Checkpoint công khai đã học trên nhãn của toàn bộ train nuScenes, gồm cả pool, nên đó là leakage. Vì vậy phải huấn luyện từ đầu trên S.
Vì sao k-NN phải tìm ở scene khác?	Frame liền nhau trong một scene giống hệt nhau, nên láng giềng cùng scene làm mọi frame trông “không hiếm”.
Vì sao chia $\sqrt{d}$ mỗi khối?	Để khối BEV 2500 chiều không lấn át khối cảm biến 3 chiều.
Vì sao gộp bằng hạng mà MMR lại dùng min-max?	Hạng tốt để cộng các tín hiệu khác thang. Min-max giữ biên độ, để MMR không loại nhầm frame hiếm vượt trội. Cả hai phương án đều được báo cáo.
Entropy và Inconsistency khác nhau thế nào?	Entropy đo độ phân vân trên từng box . Inconsistency đo việc model có trả lời giống nhau khi đầu vào chỉ bị soi gương hay không.
Sao không chọn frame model đoán sai (Hard)?	Muốn biết đoán sai thì phải có nhãn, mà nhãn của pool là thứ ta đang muốn tiết kiệm.
Tầng 1 đã có kết quả chưa?	Chưa. Code và Docker đã sẵn, đang chờ GPU để huấn luyện trên seed. Các số hiện tại đều là của Tầng 0.

Tài liệu tham khảo

- Caesar et al. *nuScenes: A multimodal dataset for autonomous driving*. CVPR 2020.
- Lang et al. *PointPillars: Fast Encoders for Object Detection from Point Clouds*. CVPR 2019.
- Zhu et al. *Class-balanced Grouping and Sampling for Point Cloud 3D Object Detection (CBGS)*. arXiv 2019.
- Peri et al. *Towards Long-Tailed 3D Detection (LT3D)*. CoRL 2022.
- Ramaswamy, Rastogi, Shim. *Efficient Algorithms for Mining Outliers from Large Data Sets*. SIGMOD 2000.
- Fischler & Bolles. *Random Sample Consensus (RANSAC)*. Communications of the ACM, 1981.
- Ester et al. *A Density-Based Algorithm for Discovering Clusters (DBSCAN)*. KDD 1996.
- Carbonell & Goldstein. *The Use of MMR, Diversity-Based Reranking*. SIGIR 1998.
- Sener & Savarese. *Active Learning for CNNs: A Core-Set Approach*. ICLR 2018.
- Gao et al. *Consistency-based Semi-supervised Active Learning*. ECCV 2020.
- OpenPCDet: bộ công cụ phát hiện 3D mã nguồn mở (cấu hình `cbgs_pp_multihead`).